

# 1. Executive Summary

**The Problem:** Kenyan farmers face two critical barriers to productivity:

1. **Global Warming:** Traditional planting seasons are shifting. "Long rains" are no longer guaranteed in March, leading to crop failure when relying on old calendars.
2. **Lack of Soil Data:** Soil testing labs are expensive and inaccessible for small-scale farmers, leading to blind planting without knowledge of soil nutrient deficiencies.

**The Solution:** A web-based "Smart Advisor" that utilizes Machine Learning to recommend the optimal crop. Unlike static systems, this application integrates **real-time weather forecasting** to adapt to climate shifts and uses a **visual soil heuristic** to estimate soil health without requiring a lab test.

---

## 2. System Architecture

The system utilizes a **3-Tier Architecture** designed for low-bandwidth environments:

### Tier 1: The Client (User Interface)

- **Technology:** React, Tailwind CSS
- **Input Features:**
  - **Hybrid Location:** Users can use "GPS Auto-Detect" (if on farm) or "Manual Search" (typing Village/Trading Center if remote).
  - **Visual Soil Input:** Instead of entering chemical numbers, users select physical soil traits (e.g., "Red Clay", "Black Cotton").

### Tier 2: The Server (Backend Logic)

- **Technology:** Python (Flask).
- **Core Modules:**
  - **data\_fetcher.py:** Handles API calls and heuristic conversions.
  - **app.py:** Manages routes and connects the frontend to the AI model.
- **AI Engine:** Scikit-Learn Decision Tree Classifier (Serialized as `crop_recommendation_model.pkl`).

### Tier 3: External Data Sources (APIs)

- **Nominatim (OpenStreetMap):** Converts location names (e.g., "Kevote") into precise Coordinates (Lat/Lon).

- **Open-Meteo API:** Provides **Real-Time & Forecasted Weather** (Rainfall, Temperature) to detect current climate conditions, ignoring historical averages if they don't match reality.

---

## 3. Key Methodologies (The "Smart" Logic)

### A. The "Visual Heuristic" for Soil Nutrients

- **Challenge:** The AI model requires Nitrogen (N), Phosphorus (P), Potassium (K), and pH for 90%+ accuracy, but farmers do not have this data.
- **Innovation:** We implemented a **Proxy Mapping System**.
  - The user selects: *"Red Volcanic Soil"*
  - The Backend maps this to a Kenyan regional average: `{'N': 60, 'P': 20, 'K': 40, 'pH': 5.5}`.
  - *Result:* We maintain high model accuracy without forcing the user to pay for a lab test.

### B. The "Climate Deviation Engine"

- **Challenge:** Historical data says March is wet, but Climate Change might make it dry.
- **Innovation:** The system queries the **Open-Meteo Forecast**.
- **Logic:** If the 30-day forecast shows rainfall is **< 50%** of the historical average, the system dynamically downgrades the rainfall input fed to the model. This forces the AI to recommend drought-resistant crops (e.g., Green Grams) instead of water-intensive ones (e.g., Maize).

### C. Nutrient Gap Analysis (Post-Processing)

- **Feature:** The system doesn't just predict a crop; it advises on soil health.
- **Logic:** After predicting "Maize", the backend checks `nutrient_requirements.json`.
  - *Example:* Maize needs 78 N. Your soil has 30 N.
  - *Output:* "Recommendation: Maize. **Warning:** Critical Nitrogen Deficit (-48). Apply Manure or Urea."

---

## 4. Data Pipeline & Model Training

### Data Sources

We merged two datasets to create `High_Accuracy_Crop_Data.csv`:

1. **Crop Recommendation Dataset:** provided crop diversity (22 unique crops).
2. **SmartGrow Dataset:** provided local context (Season Duration, specific Soil Types).

## Pre-Processing

- **Normalization:** Converted all text inputs (e.g., "Clay Loam") into numerical vectors using **One-Hot Encoding**.
- **Feature Selection:** Trained on 9 features: [Nitrogen, Phosphorus, Potassium, Temperature, Humidity, pH, Rainfall, Season\_Duration, Soil\_Type].

## Model Choice

- **Algorithm:** Decision Tree Classifier.
- **Optimization:** Tuned using **GridSearchCV** (5-Fold Cross-Validation).
- **Why this model?** Agriculture requires **interpretability**. Decision Trees allow us to trace exactly *why* a crop was chosen (e.g., "Chosen because Rainfall > 1000mm"), which is essential for explaining results to farmers.

---

## 5. Panel Defense Q&A (Cheat Sheet)

**Q1: "How can you recommend crops without a real soil test?"**

**Answer:** "I use a '**Heuristic Proxy**' method. Since 99% of farmers can't afford lab tests, I map visual soil characteristics (which farmers know) to scientific chemical profiles (based on KALRO regional data). This balances scientific accuracy with user accessibility."

**Q2: "How does your system handle Global Warming?"**

**Answer:** "Unlike static apps that rely on old calendars, my system fetches **live 14-day weather forecasts** via the Open-Meteo API. If the forecast detects a delayed rain season, the model adjusts its inputs in real-time to recommend short-season or drought-resistant crops."

**Q3: "What if the user is in a village with no GPS signal?"**

**Answer:** "The system is Hybrid. It has a **Manual Fallback Mode**. The user can type their village or nearest school name. My backend uses the Nominatim Geocoding API to find that landmark's coordinates and retrieve the relevant satellite climate data."

---

## 6. Project Deliverables

These are the core files constituting the project:

1. **app.py**: The Flask Backend (Controller).
2. **data\_fetcher.py**: The "Sensor" script (APIs for Weather/Location & Soil Heuristics).
3. **crop\_recommendation\_model.pkl**: The trained AI Brain.
4. **nutrient\_requirements.json**: The database used for the fertilizer advice.
5. **templates/index.html**: The Frontend interface.
6. **High\_Accuracy\_Crop\_Data.csv**: The cleaned, merged dataset used for training.